

Dataset Analysis Approach

March 20, 2025

sicor_municipal_co... .csv
16.7 MB

Analyze the provided dataset to identify its key characteristics for data-driven decision-making. Upon receiving a specific analytical question, address it strictly using the dataset. Your deliverables must include a clear, concise answer to the question and the Python code that extracts and processes the data used to derive your answer.

qwen-max-latest

To analyze the dataset and address your requirements, I will first load the dataset and examine its structure to identify key characteristics. Then, I will await your specific analytical question to provide a targeted answer with the corresponding Python code.

Step 1: Load and Inspect the Dataset

I will load the dataset using Python's `pandas` library and inspect its structure to understand its columns, data types, and any potential missing values or anomalies.

Here is the Python code to load and inspect the dataset:

```
python
1 import pandas as pd
2
3 # Load the dataset
4 file_path = 'sicor_municipal_contracts.csv'
5 data = pd.read_csv(file_path)
6
7 # Display basic information about the dataset
8 print("Dataset Overview:")
9 print(data.info())
10
11 # Display the first few rows of the dataset
12 print("\nFirst Few Rows:")
13 print(data.head())
14
15 # Check for missing values
16 print("\nMissing Values per Column:")
17 print(data.isnull().sum())
```

Step 2: Identify Key Characteristics

After running the above code, I will summarize:

1. The number of rows and columns in the dataset.
2. The names and data types of the columns.
3. Any missing or null values.
4. Potential insights into the dataset's structure and content.

Step 3: Address Your Analytical Question

Once you provide a specific analytical question, I will use the dataset to extract relevant data, process it, and deliver a clear, concise answer along with the Python code used for the analysis.

Please provide your analytical question so I can proceed with the analysis.

Through an advanced analysis, is it possible to predict which financial institutions will have the largest participation in granting commercialization investments in upcoming periods, based on their historical resource distribution and other available factors? Define three options to address this issue and select the best one.



To address your question about predicting which financial institutions will have the largest participation in granting commercialization investments in upcoming periods, we need to analyze historical data and identify patterns or trends that can inform such predictions. Below, I outline three potential approaches to tackle this problem, followed by a recommendation for the best option.

Approach 1: Time Series Analysis

- **Description** : Use time series forecasting models (e.g., ARIMA, SARIMA, or Prophet) to predict future participation levels of financial institutions based on their historical resource distribution trends.
- **Steps** :
 1. Extract historical participation data for each financial institution over time.
 2. Aggregate the data by time intervals (e.g., monthly or yearly).
 3. Apply time series models to forecast future participation levels.
 4. Rank institutions based on their predicted participation.
- **Pros** :
 - Captures temporal trends effectively.
 - Suitable for datasets with clear time-based patterns.
- **Cons** :
 - Requires sufficient historical data for each institution.
 - May not account for external factors (e.g., economic conditions or policy changes).

Approach 2: Machine Learning Classification

- **Description** : Train a classification model (e.g., Random Forest, Gradient Boosting, or Logistic Regression) to predict whether a financial institution will have "high participation" in the next period based on historical features.
- **Steps** :
 1. Define a target variable, such as "high participation" (e.g., top 20% of institutions by resource distribution).
 2. Engineer features from historical data, such as total resources distributed, average investment size, and growth rate.
 3. Split the data into training and testing sets.
 4. Train a classification model and evaluate its performance using metrics like precision, recall, and F1-score.
 5. Use the trained model to predict high-participation institutions for the next period.
- **Pros** :
 - Can incorporate multiple features beyond just time trends.
 - Flexible to include external variables if available.
- **Cons** :
 - Requires careful feature engineering.
 - Performance depends on the quality and relevance of features.

Approach 3: Network Analysis

- **Description** : Analyze the relationships between financial institutions and municipalities using network analysis techniques to identify key players likely to dominate future participation.
- **Steps** :
 1. Construct a bipartite network where nodes represent financial institutions and municipalities, and edges represent contracts or investments.
 2. Calculate centrality measures (e.g., degree centrality, betweenness centrality) to identify highly connected institutions.
 3. Analyze trends in centrality over time to predict future participation.
 4. Rank institutions based on their centrality scores.
- **Pros** :
 - Captures relational dynamics between institutions and municipalities.



- Provides insights into systemic importance.
- **Cons :**
 - Requires detailed relationship data.
 - May not directly predict participation levels.

Recommendation: Approach 2 (Machine Learning Classification)

Among the three options, **Machine Learning Classification** is the most robust and flexible approach for predicting future participation levels. It allows us to leverage both historical trends and additional contextual features, making it well-suited for datasets like the one provided. Moreover, classification models can handle complex patterns and interactions in the data, offering actionable insights for decision-making.

Implementation Plan

Below is the Python code to implement Approach 2, assuming the dataset contains relevant features such as `institution_name`, `investment_amount`, `year`, and other attributes:

```
python
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.metrics import classification_report
5
6 # Load the dataset
7 file_path = 'sicor_municipal_contracts.csv'
8 data = pd.read_csv(file_path)
9
10 # Feature Engineering
11 # Example: Aggregate historical data by institution
12 data['year'] = pd.to_datetime(data['year'], errors='coerce').dt.year # Ensure year is numeric
13 grouped = data.groupby(['institution_name', 'year']).agg({
14     'investment_amount': ['sum', 'mean', 'count']
15 }).reset_index()
16 grouped.columns = ['institution_name', 'year', 'total_investment', 'avg_investment', 'num_contracts']
17
18 # Define target variable: "High Participation" (top 20% by total investment)
19 grouped['high_participation'] = grouped.groupby('year')['total_investment'].apply(
20     lambda x: x >= x.quantile(0.8)
21 ).astype(int)
22
23 # Prepare features and target
24 features = ['total_investment', 'avg_investment', 'num_contracts']
25 X = grouped[features]
26 y = grouped['high_participation']
27
28 # Train-test split
29 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
30
31 # Train a Random Forest Classifier
32 model = RandomForestClassifier(random_state=42)
33 model.fit(X_train, y_train)
34
35 # Evaluate the model
36 y_pred = model.predict(X_test)
37 print(classification_report(y_test, y_pred))
38
39 # Predict high-participation institutions for the next period
40 # Assuming you have new data for prediction
41 # new_data = ... # New data with the same features
42 # predictions = model.predict(new_data)
```

Next Steps

1. Run the above code to train and evaluate the model.
2. Use the trained model to predict high-participation institutions for the next period.
3. Refine the model by incorporating additional features (e.g., economic indicators, policy changes) if available.

Let me know if you'd like further clarification or assistance with implementing this solution!



